ENTERPRISE KUBERNETES MASTERY

AI Platform Engineering Handbook

PART VI KUBERNETES NETWORKING

CNI, eBPF, Service Mesh, Gateway API, Multi-Cluster

Volume 6 of 16 Core Series

Prerequisites: Parts I through V

Edition 2025-2026

TABLE OF CONTENTS

CHAPTER 1

Kubernetes Networking Model

Kubernetes networking is built on a flat, unified IP model. Unlike Docker's default NAT-based networking, Kubernetes requires that every Pod gets a unique, routable IP address and that Pods can communicate with each other without NAT. This model simplifies application networking dramatically but requires careful infrastructure design.

The Four Networking Requirements

* **1. Pod-to-Pod (same node)**: Pods on the same node must be able to communicate directly using their Pod IPs without NAT. Implemented via a Linux bridge or eBPF on the node.
* **2. Pod-to-Pod (cross-node)**: Pods on different nodes must communicate using their Pod IPs without NAT. Implemented via overlay networks (VXLAN), BGP routing, or eBPF.
* **3. Pod-to-Service**: Pods must be able to reach Services by their ClusterIP or DNS name. Implemented via iptables DNAT (kube-proxy) or eBPF (Cilium).
* **4. External-to-Service**: External clients must reach Services via NodePort, LoadBalancer, Ingress, or Gateway. Implemented via cloud load balancers and node routing.

The Pod IP Model

```
Node IP space and Pod IP space are different and must not overlap: Example cluster network design: Node IPs: 192.168.1.0/24 (physical/VM network) Pod CIDR: 10.244.0.0/16 (all Pod IPs in cluster) Node-01: 10.244.0.0/24 (Pods on node-01) Node-02: 10.244.1.0/24 (Pods on node-02) Node-03: 10.244.2.0/24 (Pods on node-03) Service CIDR: 10.96.0.0/12 (all Service ClusterIPs) CRITICAL: These CIDRs must not overlap with: - Your physical/VM network - VPN routes (very common enterprise problem) - On-premises RFC 1918 ranges in use # Specify at cluster creation (kubeadm): kubeadm init \ --pod-network-cidr=10.244.0.0/16 \ --service-cidr=10.96.0.0/12 # These cannot be changed after cluster creation without rebuild
```

Network Plugin Requirements (CNI Contract)

```
Every CNI plugin must satisfy the Kubernetes network contract: 1. Each Pod gets a unique IP within the cluster 2. All Pods can communicate with all other Pods without NAT 3. Agents on a node (kubelet, system daemons) can communicate with all Pods on that node 4. The Pod IP the container sees is the same IP that others use to reach it Beyond these requirements, CNI plugins differ in: - Implementation (iptables vs eBPF vs DPDK) - Performance characteristics - NetworkPolicy support - Encryption capabilities (WireGuard, IPsec) - Multi-cluster support - Observability depth
```

CHAPTER 2

Pod Networking Deep Dive

Understanding exactly what happens at the Linux level when a Pod is created and starts communicating is essential for network troubleshooting, CNI selection, and performance optimisation.

Pod Network Namespace Setup Sequence

```
When kubelet creates a Pod, the networking is set up in this sequence: 1. containerd creates the pause (infra) container pause creates a new network namespace (netns) All other containers in Pod JOIN this netns 2. containerd calls CNI plugin ADD: CNI receives: container ID, netns path, Pod name/namespace 3. CNI creates veth pair: eth0 (inside Pod netns) <---> veth_xxxx (host netns) 4. CNI assigns Pod IP to eth0: ip addr add 10.244.1.5/32 dev eth0 (host-local IPAM) or IPAM from central pool (Cilium, Calico) 5. CNI sets up routing: Inside Pod netns: default via 169.254.1.1 dev eth0 (Calico) or: default via 10.244.1.1 dev eth0 (flannel bridge) Host netns: 10.244.1.5 via veth_xxxx (direct route to Pod) 6. Pod is now reachable at its IP from anywhere in cluster # Inspect Pod network setup on node: # Find Pod PID: crictl pods --name my-pod -q crictl inspect POD_ID | python3 -c "import json,sys; print(json.load(sys.stdin)['status']['pid'])" # Inspect from host: nsenter -t POD_PID -n ip addr show nsenter -t POD_PID -n ip route show ip link show | grep veth
```

Packet Flow: Pod-to-Pod Same Node (Bridge CNI)

```
Pod-A (10.244.1.5) -> Pod-B (10.244.1.6), SAME NODE with Linux bridge: 1. Pod-A sends: src=10.244.1.5, dst=10.244.1.6 2. Pod-A kernel: dst in 10.244.1.0/24 -> via eth0 (direct, same subnet) ARP request: who has 10.244.1.6? (Bridge responds with MAC of veth for Pod-B) 3. Packet exits eth0 -> enters host veth pair -> arrives at bridge (cni0) 4. Bridge L2 forwarding: dst MAC -> Pod-B's veth 5. Packet crosses veth pair -> arrives at Pod-B's eth0 6. Pod-B receives packet Total path: 2 veth crossings + bridge L2 lookup Latency: sub-microsecond on same host
```

Packet Flow: Pod-to-Pod Cross-Node (VXLAN)

```
Pod-A (node-01: 10.244.0.5) -> Pod-B (node-02: 10.244.1.7): VXLAN OVERLAY PATH (Flannel/Calico VXLAN mode): 1. Pod-A sends: src=10.244.0.5, dst=10.244.1.7 2. Host routing on node-01: 10.244.1.0/24 -> via flannel.1 (VXLAN interface) (flannel.1 has ARP/FDB entries: 10.244.1.7 -> node-02's MAC/IP) 3. VXLAN encapsulation: Inner packet: src=10.244.0.5, dst=10.244.1.7 Outer UDP: src=node-01:8472, dst=node-02:8472 Outer IP: src=192.168.1.10, dst=192.168.1.11 4. Physical network delivers outer UDP packet to node-02 5. node-02 flannel.1 decapsulates VXLAN 6. Inner packet routed to Pod-B via local veth Overhead: 50-byte VXLAN header per packet Requires: UDP port 8472 open between nodes DIRECT ROUTING PATH (Calico BGP / Cilium native routing): 1. Pod-A sends: src=10.244.0.5, dst=10.244.1.7 2. Host routing on node-01 (BGP learned route): 10.244.1.0/24 -> via 192.168.1.11 (node-02 IP) 3. Packet routed directly across physical network 4. node-02 receives, routes to Pod-B via local veth No encapsulation overhead; requires L2 adjacency or BGP peer
```

CHAPTER 3

CNI Architecture and Plugin Selection

The Container Network Interface (CNI) is a specification and set of libraries for writing plugins to configure network interfaces in Linux containers. When a Pod is created or deleted, kubelet calls the CNI plugin to set up or tear down networking.

CNI Plugin Architecture

```
CNI plugin execution flow: kubelet | | executes CNI plugin binary with: | -
CNI_COMMAND=ADD|DEL|CHECK | - CNI_CONTAINERID= | - CNI_NETNS=/proc/PID/ns/net | -
CNI_IFNAME=eth0 | - CNI_ARGS=K8S_POD_NAME=... | + stdin: JSON network config | CNI plugin
binary (e.g., /opt/cni/bin/calico) Performs network setup Returns JSON result with assigned
IPs # CNI config location: ls /etc/cni/net.d/ cat /etc/cni/net.d/10-calico.conflist # CNI
binaries: ls /opt/cni/bin/
```

CNI Plugin Comparison Matrix

Feature	Calico	Cilium	Flannel	Antrea	WeaveNet
Data plane	iptables or eBPF	eBPF (primary)	iptables	iptables or OVS	iptables
kube-proxy replacement	Yes (eBPF mode)	Yes (KPR mode)	No	Partial	No
NetworkPolicy	Full (K8s + custom)	Full (K8s + custom L7)	None	Full	Full
FQDN policies	Yes	Yes	No	No	No
L7 policy (HTTP)	Via Envoy integration	Native eBPF	No	No	No
Encryption	WireGuard, IPsec	WireGuard, IPsec	No	IPsec	Yes
Multi-cluster	Calico Federation	Cluster Mesh, Hubble	No	Multi-cluster SVC	No
Overlay	VXLAN, IPIP	VXLAN, Geneve	VXLAN	Geneve	VXLAN
Direct routing	BGP, cross-subnet	Native routing	No	No	No
Observability	Flow logs	Hubble (L7 flows)	Basic	Theia	Basic

Feature	Calico	Cilium	Flannel	Antrea	WeaveNet
Scale (nodes)	5,000+	10,000+	500	1,000+	200
Best for	BGP environments, on-prem	Large scale, eBPF, AI	Simple clusters	VMware/OVS	Dev/test

CNI Selection Guide for Enterprise

* **Greenfield cloud-native cluster at scale**: Cilium with eBPF kube-proxy replacement. Best performance, richest observability via Hubble, native L7 policy, WireGuard encryption.
* **On-premises with BGP infrastructure**: Calico with BGP mode. Native routing without overlay, integrates with existing network fabric, mature enterprise support.
* **Simple cluster (dev, edge, small production)**: Flannel for simplicity, or Calico for NetworkPolicy support without complexity.
* **OpenShift / Red Hat environment**: OVN-Kubernetes (default in OpenShift 4.x), or Calico.
* **AI/ML cluster with high network throughput**: Cilium with native routing + RDMA-capable NICs for GPU-to-GPU communication; consider SR-IOV for network-intensive workloads.

CHAPTER 4

Calico -- Policy-First Networking

Calico (Tigera) is the most widely deployed Kubernetes CNI, particularly in enterprise on-premises environments. It supports multiple data planes (iptables, eBPF, VPP) and offers rich NetworkPolicy with BGP-based routing for zero-overhead direct pod networking.

Calico Architecture

```
Calico components: calico-node DaemonSet (on every node): +-- Felix: programs iptables/eBPF rules, routes, ARP entries +-- BIRD: BGP daemon (advertises Pod CIDR routes to peers) +-- confd: watches etcd/K8s API, updates BIRD config calico-kube-controllers Deployment: Syncs K8s resources to Calico datastore calico-apiserver (optional): Serves Calico-specific CRDs via K8s API aggregation Typha (optional, for large clusters): Fanout proxy between Calico datastore and Felix instances Reduces datastore load for 50+ node clusters Data planes: Standard: iptables (default, most compatible) eBPF: Replaces iptables with eBPF maps (better scale) VPP: DPDK-based (ultra-high performance, specialist)
```

Calico BGP Configuration

```
# Default BGP configuration (full mesh between nodes): apiVersion: projectcalico.org/v3 kind: BGPConfiguration metadata: name: default spec: logSeverityScreen: Info nodeToNodeMeshEnabled: true # Full mesh (default, OK up to ~50 nodes) asNumber: 64512 # For larger clusters: route reflectors apiVersion: projectcalico.org/v3 kind: BGPPeer metadata: name: bgppeer-rr spec: nodeSelector: has(is-route-reflector) # Route reflector nodes peerIP: 192.168.1.10 asNumber: 64512 # Upstream BGP peer (top-of-rack switch): apiVersion: projectcalico.org/v3 kind: BGPPeer metadata: name: bgppeer-tor spec: peerIP: 192.168.1.1 asNumber: 64500 nodeSelector: all() # All nodes peer with ToR switch
```

Calico GlobalNetworkPolicy

Calico extends Kubernetes NetworkPolicy with GlobalNetworkPolicy (cluster-scoped) and richer selector syntax:

```
# Calico GlobalNetworkPolicy -- applies across all namespaces: apiVersion: projectcalico.org/v3 kind: GlobalNetworkPolicy metadata: name: deny-egress-to-metadata spec: order: 100 selector: all() types: [Egress] egress: - action: Deny destination: nets: [169.254.169.254/32] # Block cloud metadata endpoint --- # FQDN-based policy (Calico Enterprise): apiVersion: projectcalico.org/v3 kind: NetworkPolicy metadata: name: allow-api-egress namespace: production spec: selector: app == 'myapp' types: [Egress] egress: - action: Allow destination: domains: ['api.openai.com', '*.anthropic.com']
```

CHAPTER 5

Cilium -- eBPF-Native Networking

Cilium uses eBPF as its primary data plane, replacing iptables for service routing, network policy enforcement, and load balancing. This enables significantly better performance, richer observability (Hubble), and L7-aware policies at scale. Cilium is the recommended CNI for large-scale enterprise and AI workloads.

Cilium Architecture

```
Cilium components: cilium DaemonSet (on every node): +-- Cilium Agent: manages eBPF programs, policies, endpoints +-- eBPF programs: loaded into kernel at XDP, TC, socket hooks +-- BPF maps: store endpoint state, policy, conntrack, LB tables +-- Envoy (optional): L7 proxy for HTTP/gRPC policy and observability cilium-operator Deployment: Manages IPAM (IP address assignment), KVStore sync Hubble (observability): hubble-relay: aggregates flow data from all nodes hubble-ui: web interface for network flow visualisation eBPF programs by hook point: XDP (NIC driver level): NodePort/LoadBalancer packet processing Direct Server Return (DSR) DDoS protection TC (Traffic Control): Endpoint-to-endpoint routing Policy enforcement Connection tracking Socket hooks: Service routing without packet looping Transparent proxy insertion
```

Cilium kube-proxy Replacement

Cilium's most significant capability is replacing kube-proxy entirely with eBPF, eliminating iptables from the Service routing path:

```
# Install Cilium with full kube-proxy replacement: helm repo add cilium https://helm.cilium.io/ helm install cilium cilium/cilium \ --namespace kube-system \ --set kubeProxyReplacement=true \ --set k8sServiceHost=API_SERVER_IP \ --set k8sServicePort=6443 \ --set loadBalancer.mode=dsr \ --set encryption.enabled=true \ --set encryption.type=wireguard \ --set hubble.enabled=true \ --set hubble.relay.enabled=true \ --set hubble.ui.enabled=true # Verify kube-proxy replacement: cilium status | grep KubeProxyReplacement # Performance comparison (kube-proxy vs Cilium eBPF): # Service lookup: iptables O(n) -> eBPF O(1) # 10,000 services: iptables ~10ms overhead vs eBPF ~1us # Conntrack: kernel conntrack vs eBPF custom conntrack # CPU overhead: 30-40% less CPU for network processing at scale
```

Hubble Network Observability

Hubble provides real-time, L3-L7 network flow visibility across the entire cluster without any application instrumentation:

```
# Hubble CLI (installed separately): # Observe all network flows in real time: hubble observe --follow # Filter: flows to/from a specific Pod: hubble observe --pod production/api-server-abc # Filter: only dropped flows (policy violations): hubble observe --verdict DROPPED # Filter: HTTP flows with status codes: hubble observe --protocol http --follow # Service dependency map: hubble observe --output json | \ jq -r '[.source.namespace,.source.pod_name,.destination.namespace,.destination.pod_name] | @csv' # Hubble metrics in Prometheus: # hubble_flows_processed_total # hubble_drop_total (by reason, namespace, direction) # hubble_tcp_flags_total # hubble_http_requests_total (L7 visibility)
```

Cilium L7 Network Policy

Cilium can enforce HTTP/gRPC policy at L7 using its embedded Envoy proxy or eBPF-native HTTP inspection:

```
apiVersion: cilium.io/v2 kind: CiliumNetworkPolicy metadata: name: api-l7-policy namespace: production spec: endpointSelector: matchLabels: { app: api-server } ingress: - fromEndpoints: - matchLabels: { app: frontend } toPorts: - ports: - port: '8080' protocol: TCP rules: http: - method: GET path: /api/v1/.* - method: POST path: /api/v1/inference headers: - X-API-Key: .*
```

CHAPTER 6

Flannel and Other CNIs

Flannel is the simplest widely-used CNI. It provides basic overlay networking without NetworkPolicy support. Suitable for development clusters, edge deployments, and simple production environments where Calico or Cilium capabilities are not needed.

Flannel Backends

Backend	Mechanism	Performance	Requirements
VXLAN	UDP encapsulation (L2 over L3)	Good; ~10-15% overhead	UDP 8472 between nodes
host-gw	Direct routing via host gateway	Best; zero overhead	Requires L2 adjacency (same subnet)
WireGuard	Encrypted VXLAN	Good; encryption overhead	WireGuard kernel module
UDP (deprecated)	Userspace UDP	Poor; avoid	Legacy only

Other Notable CNIs

* **Antrea**: VMware-originated; OVS-based; integrates with NSX; Octant UI.
* **OVN-Kubernetes**: OpenVSwitch + OVN; default in OpenShift; good for VMware environments.
* **Multus**: Meta-CNI enabling multiple CNI plugins per Pod; used for SR-IOV + primary CNI.
* **Whereabouts**: IPAM plugin for static IP management across clusters.
* **SR-IOV CNI**: Hardware NIC virtualisation; near wire-speed networking for AI/HPC workloads.
* **DPDK/VPP**: Kernel-bypass networking for extreme throughput (100Gbps+); specialist use.

CHAPTER 7

Service Networking Internals

Understanding exactly how Service ClusterIPs work at the Linux level is essential for troubleshooting connectivity issues and understanding performance characteristics. The implementation differs significantly between iptables, IPVS, and eBPF modes.

iptables Service Implementation

```
iptables chains created by kube-proxy for a Service with 3 endpoints: Service: my-svc (ClusterIP: 10.96.50.100:80 -> Pods: 10.244.1.5, 10.244.2.3, 10.244.3.9) PREROUTING -t nat: -> KUBE-SERVICES -> match 10.96.50.100:80 -> KUBE-SVC-XXXXXXXX -> 33.3% probability: KUBE-SEP-AAAA (DNAT to 10.244.1.5:8080) -> 50.0% probability: KUBE-SEP-BBBB (DNAT to 10.244.2.3:8080) -> 100% probability: KUBE-SEP-CCCC (DNAT to 10.244.3.9:8080) # View Service iptables rules: iptables -t nat -L KUBE-SERVICES -n | grep my-svc iptables -t nat -L KUBE-SVC-XXXXXXXX -n iptables-save -t nat | grep KUBE-SVC # Performance implication: O(n) linear scan # 10,000 services = up to 200,000 iptables rules per packet path # Each iptables rule evaluation: ~1-2 microseconds # At 10K services: 200-400ms added latency -> severe for high-rate traffic
```

IPVS Service Implementation

```
IPVS (IP Virtual Server) uses kernel hash tables for O(1) Service lookup: kube-proxy creates virtual server entries in IPVS: TCP 10.96.50.100:80 rr -> 10.244.1.5:8080 weight=1 -> 10.244.2.3:8080 weight=1 -> 10.244.3.9:8080 weight=1 Load balancing algorithms: rr: Round-robin (default) lc: Least connections dh: Destination hash (session affinity) sh: Source hash (client IP affinity) sed: Shortest expected delay nq: Never queue # Enable IPVS mode (kube-proxy config): mode: ipvs ipvs: scheduler: rr strictARP: true # Required for MetalLB # Inspect IPVS tables: ipvsadm -Ln | grep -A5 10.96.50.100
```

Cilium eBPF Service Implementation

```
Cilium replaces IPVS and iptables with eBPF maps at the socket level: BPF map: SERVICE_MAP Key: (10.96.50.100, port=80, proto=TCP) Value: [backend_id_1, backend_id_2, backend_id_3] BPF map: BACKEND_MAP Key: backend_id_1 Value: (10.244.1.5, port=8080) Socket-level hook (BPF_PROG_TYPE_CGROUP_SOCK_ADDR): When app calls connect(10.96.50.100:80): eBPF hook intercepts at socket creation Looks up service in BPF map (O(1) hash) Rewrites destination to backend IP:port No packet encap/decap; destination rewrite at socket level Connection goes directly to backend Pod IP Result: - No iptables traversal - No conntrack overhead for established connections - O(1) lookup regardless of service count - Source IP preserved (no SNAT for same-node backends) # Inspect Cilium BPF service map: cilium service list cilium bpf lb list
```

CHAPTER 8

CoreDNS Advanced Patterns

CoreDNS is the DNS backbone of every Kubernetes cluster. Advanced configurations include forwarding to internal DNS, stub zones for external services, response caching tuning, and external DNS for automatic DNS record management.

Topology-Aware DNS

```
# External DNS -- automatically create DNS records for Services/Ingress: # When you create:
Service type=LoadBalancer with IP 35.1.2.3 # ExternalDNS creates: api.example.com -> 35.1.2.3
in Route53/Cloud DNS # Install External DNS: helm repo add external-dns
https://kubernetes-sigs.github.io/external-dns/ helm install external-dns
external-dns/external-dns \ --set provider=aws \ --set aws.zoneType=public \ --set
txtOwnerId=my-cluster # Annotate Service for DNS record creation: metadata: annotations:
external-dns.alpha.kubernetes.io/hostname: api.example.com
external-dns.alpha.kubernetes.io/ttl: '60'
```

CoreDNS Performance Tuning

```
# CoreDNS ConfigMap tuning for large clusters: .:53 { errors health { lameduck 5s } ready
kubernetes cluster.local in-addr.arpa ip6.arpa { pods insecure fallthrough in-addr.arpa
ip6.arpa ttl 30 } # Increase cache (reduce upstream queries): cache { success 9984 30 # 9984
positive entries, 30s TTL denial 9984 5 # 9984 negative entries, 5s TTL } # Prefer UDP (avoid
TCP handshake): forward . /etc/resolv.conf { prefer_udp max_concurrent 1000 } # Autopath
(reduce search domain queries): autopath @kubernetes prometheus :9153 reload loadbalance } #
Scale CoreDNS replicas for large clusters: # Rule of thumb: 1 CoreDNS replica per 500 nodes
kubectl scale deployment coredns -n kube-system --replicas=4 # NodeLocal DNSCache -- cache at
node level: # Reduces CoreDNS load by 80%+ in large clusters # DaemonSet that runs a local DNS
cache on each node kubectl apply -f https://raw.githubusercontent.com/kubernetes/kubernetes/ma
ster/cluster/addons/dns/nodelocaldns/nodelocaldns.yaml
```

CHAPTER 9

Ingress Controllers in Production

An Ingress controller implements the Ingress resource, providing HTTP routing, TLS termination, and load balancing for external traffic entering the cluster. The Ingress resource is controller-agnostic; the controller implements the behaviour.

Ingress Controller Comparison

Controller	Proxy	Strengths	Scale	Use Case
ingress-nginx	Nginx	Mature, widespread, rich annotations	Large	General purpose production
HAProxy Ingress	HAProxy	High performance, fine-grained TCP	Very large	High-traffic, complex routing
Traefik	Traefik	Auto-discovery, Let's Encrypt, dashboard	Medium	Dev-friendly, dynamic configs
AWS ALB Ingress	AWS ALB	Native AWS integration, WAF, Shield	Very large	AWS-native workloads
Istio Gateway	Envoy	Full service mesh, mTLS, telemetry	Large	Service mesh environments
Cilium Gateway	eBPF+Envoy	eBPF performance, Gateway API native	Very large	Cilium clusters
Kong	Nginx+Kong	API gateway features (auth, rate limit, plugins)	Large	API management

ingress-nginx Production Configuration

```
# Install ingress-nginx with production settings: helm repo add ingress-nginx
https://kubernetes.github.io/ingress-nginx helm install ingress-nginx
ingress-nginx/ingress-nginx \ --namespace ingress-nginx --create-namespace \ --set
controller.replicaCount=3 \ --set controller.minAvailable=2 \ --set
controller.resources.requests.cpu=100m \ --set controller.resources.requests.memory=256Mi \
--set controller.resources.limits.memory=512Mi \ --set controller.metrics.enabled=true \ --set
controller.podAnnotations.prometheus\.io/scrape=true \ --set
controller.config.use-forwarded-headers=true \ --set
controller.config.compute-full-forwarded-for=true \ --set
controller.config.use-proxy-protocol=false # Key ConfigMap settings for production: data:
keep-alive: '75' keep-alive-requests: '100' upstream-keepalive-connections: '32'
proxy-connect-timeout: '5' proxy-send-timeout: '60' proxy-read-timeout: '60'
log-format-escape-json: 'true' log-format-upstream:
'{"time":"$time_iso8601","status":"$status",...}'
```

CHAPTER 10

Gateway API -- Advanced Patterns

Gateway API is the next generation Kubernetes networking API, graduating to stable (v1) in Kubernetes 1.28. It addresses the limitations of Ingress: role-based management, richer routing semantics, protocol support beyond HTTP, and extensibility.

Gateway API Resource Hierarchy

```
ROLE: Infrastructure Provider (cloud team) GatewayClass: Defines the type of Gateway (which controller implements it) apiVersion: gateway.networking.k8s.io/v1 kind: GatewayClass metadata: { name: cilium } spec: controllerName: io.cilium/gateway-controller ROLE: Cluster Operator (platform team) Gateway: Deploys a load balancer + listener configuration apiVersion: gateway.networking.k8s.io/v1 kind: Gateway metadata: { name: prod-gateway, namespace: infra } spec: gatewayClassName: cilium listeners: - name: https port: 443 protocol: HTTPS tls: { certificateRefs: [{name: wildcard-cert}] } allowedRoutes: { namespaces: { from: Selector } } ROLE: Application Developer (app team) HTTPRoute: Defines routing rules for their application apiVersion: gateway.networking.k8s.io/v1 kind: HTTPRoute metadata: { name: api-routes, namespace: production } spec: parentRefs: [{name: prod-gateway, namespace: infra}] hostnames: [api.example.com] rules: - matches: [{path: {type: PathPrefix, value: /api}}] backendRefs: - name: api-service port: 80 weight: 90 - name: api-canary port: 80 weight: 10
```

Gateway API for AI Serving

Gateway API with traffic splitting is ideal for LLM inference canary deployments:

```
# Canary deployment: route 5% of requests to new LLM version apiVersion: gateway.networking.k8s.io/v1 kind: HTTPRoute metadata: name: llm-api-routing namespace: ai-serving spec: parentRefs: [{name: ai-gateway}] hostnames: [llm.internal.corp] rules: # Header-based routing for testing: - matches: - headers: - name: X-Model-Version value: v2 backendRefs: [{name: llm-v2, port: 80}] # Weighted split for canary: - backendRefs: - name: llm-v1 port: 80 weight: 95 - name: llm-v2 port: 80 weight: 5 filters: - type: ResponseHeaderModifier responseHeaderModifier: add: - name: X-Served-By value: llm-gateway
```

CHAPTER 11

Service Mesh: Istio Deep Dive

A service mesh adds a transparent infrastructure layer for service-to-service communication, providing: mutual TLS (mTLS) for encryption and authentication, observability (traces, metrics, logs), traffic management (retries, timeouts, circuit breaking, canary deployments), and policy enforcement -- all without changing application code.

Istio Architecture

```
Istio components: CONTROL PLANE: istiod Pilot: Service discovery; push xDS config to proxies Citadel: Certificate authority; issue workload certificates (SPIFFE) Galley: Configuration validation and distribution DATA PLANE: Envoy sidecar proxies One Envoy sidecar per Pod (injected automatically) All inbound/outbound traffic intercepted via iptables rules Envoy enforces: mTLS, policies, retries, timeouts, circuit breaking Envoy exports: request metrics, distributed traces (Jaeger/Zipkin) Traffic interception (per Pod): iptables -t nat -A PREROUTING -p tcp -j ISTIO_INBOUND iptables -t nat -A OUTPUT -p tcp -j ISTIO_OUTPUT All TCP traffic redirected to Envoy (port 15001 egress, 15006 ingress) xDS configuration protocol: istiod pushes configuration to all Envoy proxies via gRPC streaming LDS (Listeners), RDS (Routes), CDS (Clusters), EDS (Endpoints)
```

Istio Traffic Management

```
# VirtualService -- traffic routing rules: apiVersion: networking.istio.io/v1 kind: VirtualService metadata: name: api-routing spec: hosts: [api-service] http: # Header-based routing: - match: - headers: X-API-Version: { exact: v2 } route: - destination: { host: api-service, subset: v2 } # Canary with retries: - route: - destination: { host: api-service, subset: v1 } weight: 90 - destination: { host: api-service, subset: v2 } weight: 10 retries: attempts: 3 perTryTimeout: 2s retryOn: gateway-error,connect-failure,refused-stream timeout: 10s fault: delay: percentage: { value: 0.1 } fixedDelay: 5s # Inject 5s delay to 0.1% of requests (chaos testing) # DestinationRule -- load balancing and connection pool: apiVersion: networking.istio.io/v1 kind: DestinationRule metadata: name: api-destination spec: host: api-service trafficPolicy: connectionPool: tcp: { maxConnections: 100 } http: http2MaxRequests: 1000 pendingRequests: 100 outlierDetection: consecutive5xxErrors: 5 interval: 10s baseEjectionTime: 30s subsets: - name: v1 labels: { version: v1 } - name: v2 labels: { version: v2 }
```

CHAPTER 12

Linkerd and Ambient Mesh

Linkerd -- Lightweight Service Mesh

Linkerd focuses on simplicity, performance, and security. Unlike Istio's Envoy-based heavy sidecar, Linkerd uses a micro-proxy written in Rust (linkerd2-proxy) that is much smaller and faster:

Dimension	Istio	Linkerd
Proxy	Envoy (C++)	linkerd2-proxy (Rust)
Proxy memory	50-200MB per Pod	10-30MB per Pod
Proxy CPU	50-200m per Pod	5-30m per Pod
Configuration complexity	High (many CRDs)	Low (simple CRDs)
L7 protocol	HTTP/1, HTTP/2, gRPC, WebSocket	HTTP/1, HTTP/2, gRPC
Traffic management	Very rich (VirtualService, DR)	Basic (HTTPRoute, TrafficSplit)
Multi-cluster	Yes (complex)	Yes (simpler)
mTLS	Yes (cert-manager or Citadel)	Yes (automatic, cert-manager)
Best for	Complex traffic management, full control	Simplicity, low overhead, Rust-safe

Istio Ambient Mesh -- Sidecar-Free Architecture

Ambient Mesh (Istio 1.21+, stable 2025) eliminates the sidecar model entirely, moving mesh functionality to the node level and a dedicated waypoint proxy layer. This reduces resource overhead dramatically and removes the Pod restart requirement for mesh adoption:

```
Ambient Mesh architecture: LAYER 1: ztunnel (per-node DaemonSet) Lightweight Rust proxy on each node Provides: mTLS, L4 telemetry, L4 policy Zero sidecar: all Pods on node participate automatically Traffic interception: eBPF (redirects traffic to ztunnel) LAYER 2: Waypoint Proxy (per-namespace/service, Envoy-based) Optional; only needed for L7 features Provides: HTTP routing, header manipulation, L7 policy Deployed only for services requiring L7 capabilities Benefits over sidecar: - No Pod restart required to join mesh - 90% lower memory overhead vs sidecar - Shared ztunnel: no per-Pod proxy overhead - Simpler upgrades: update ztunnel DaemonSet, not every Pod # Enable Ambient Mesh: helm install istio-cni istio/cni -n istio-system \ --set profile=ambient helm install ztunnel istio/ztunnel -n istio-system # Opt namespace into ambient mesh: kubectl label namespace production istio.io/dataplane-mode=ambient
```

CHAPTER 13

East-West and North-South Traffic Patterns

Traffic Flow Classification

Pattern	Direction	Examples	Primary Controls
North-South (inbound)	Internet -> Cluster	User HTTP/HTTPS requests	Ingress, Gateway, LB, WAF
North-South (outbound)	Cluster -> Internet	API calls to external SaaS	Egress gateway, NetworkPolicy, proxy
East-West (in-cluster)	Pod <-> Pod	Service-to-service, DB calls	Service, NetworkPolicy, mTLS
East-West (cross-cluster)	Cluster <-> Cluster	Multi-region failover	Cluster Mesh, Federation

Egress Control Patterns

Controlling outbound traffic from Pods prevents data exfiltration and enforces compliance. Multiple layers provide defence in depth:

* **NetworkPolicy egress rules**: Block all egress by default; allow only specific destinations. Kubernetes-native but limited to IP/port (no FQDN).
* **Calico FQDN-based egress**: Allow egress to api.openai.com without knowing IPs. Calico Enterprise resolves FQDN to IPs dynamically and enforces accordingly.
* **Cilium FQDN policy**: Same as Calico; native in open-source Cilium.
* **Istio Egress Gateway**: Route all external traffic through a dedicated Egress Gateway Pod. Enables centralised logging, access control, and TLS origination for external calls.
* **HTTP proxy (Squid/Envoy)**: Traditional HTTP CONNECT proxy. Some enterprises require all outbound HTTP to traverse a proxy for DLP scanning.

CHAPTER 14

Multi-Cluster Networking

Multi-cluster networking extends Kubernetes Services and networking across multiple clusters. This is essential for global deployments, disaster recovery, regulatory data locality, and AI platform architectures requiring model serving in multiple regions.

Multi-Cluster Approaches

Approach	Mechanism	Latency	Complexity	Best For
Cilium Cluster Mesh	Direct Pod-to-Pod routing	Low (direct)	Medium	Same-org, trusted clusters
Istio Multi-Cluster	Gateway-based service mirroring	Medium (via GW)	High	Service mesh environments
Submariner	IPsec tunnels between clusters	Medium	Medium	Any CNI, on-premises
Admiral	Istio service federation	Medium	High	Large Istio deployments
DNS-based (external-dns)	DNS round-robin/failover	High (DNS TTL)	Low	Simple failover, CDN
Service API (Multi-Cluster SIGs)	Standard API (emerging)	Varies	Low	Future standard

Cilium Cluster Mesh

Cilium Cluster Mesh enables transparent cross-cluster service discovery and direct Pod-to-Pod communication between clusters sharing the same Cilium installation:

```
# Connect two clusters with Cilium Cluster Mesh: # On cluster-1: cilium clustermesh enable
--service-type LoadBalancer cilium clustermesh status # On cluster-2: cilium clustermesh
enable --service-type LoadBalancer # Connect cluster-2 to cluster-1: cilium clustermesh
connect \ --context cluster-1 \ --destination-context cluster-2 # Export Service from
cluster-1 to cluster-2: # Annotate Service in cluster-1: kubectl annotate service my-api \
service.cilium.io/global='true' # Service now reachable from cluster-2 as
'my-api.namespace.svc.cluster.local' # Traffic distributed across endpoints in BOTH clusters #
Prefer local cluster (failover to remote if local unavailable): kubectl annotate service
my-api \ service.cilium.io/shared=false service.cilium.io/global=true
```

CHAPTER 15

Network Observability

Network observability in Kubernetes spans from basic connectivity metrics to full L7 flow visibility. The right observability stack depends on your CNI choice and observability requirements.

Observability Stack by CNI

CNI	L3/L4 Flows	L7 Visibility	Tool
Cilium	Native Hubble	Native Hubble (HTTP, gRPC, DNS)	Hubble CLI + UI, Prometheus
Calico	Flow logs	Via sidecar (Istio/Linkerd)	Calico flow logs, Kibana
Any CNI	Network metrics	Via service mesh	Prometheus + Grafana
Any CNI	eBPF tracing	Syscall-level	Pixie, BPFTrace

Key Network Metrics to Monitor

* **Pod network throughput**: `container_network_transmit_bytes_total, container_network_receive_bytes_total`
* **DNS latency and errors**: `coredns_dns_request_duration_seconds, coredns_dns_responses_total`
* **Service latency**: `istio_request_duration_milliseconds (Istio), hubble_http_request_duration (Cilium)`
* **Connection errors**: `container_network_transmit_errors_total, net_conntrack_entries (conntrack table)`
* **iptables/eBPF drops**: `node_netstat_IpExt_InOctets, cilium_drop_count_total`
* **DNS lookup failures**: `coredns_dns_responses_total where rcode=SERVFAIL,NXDOMAIN`

CHAPTER 16

Network Troubleshooting Playbook

Issue: Pod cannot reach another Pod

```
Step 1: Verify Pod IPs and routing kubectl get pod -o wide # Check Pod IPs nsenter -t POD_PID -n ip route show Step 2: Test connectivity from source Pod kubectl exec -it source-pod -- ping DEST_IP kubectl exec -it source-pod -- curl -v http://DEST_IP:PORT Step 3: Check NetworkPolicy (if any) kubectl get networkpolicies -A # Check if policies select source or dest Pod Step 4: CNI-specific Cilium: cilium monitor --from-pod source-pod Calico: kubectl logs -n kube-system -l k8s-app=calico-node
```

Issue: Service not resolving

```
Step 1: Verify DNS configuration kubectl exec -it test-pod -- cat /etc/resolv.conf kubectl exec -it test-pod -- nslookup kubernetes.default Step 2: Check CoreDNS pods kubectl get pods -n kube-system -l k8s-app=kube-dns kubectl logs -n kube-system -l k8s-app=kube-dns Step 3: Verify Service and Endpoints kubectl get svc my-service kubectl get endpoints my-service # Empty endpoints = selector does not match any Ready Pods
```

Issue: High network latency

```
Step 1: Baseline with iperf3 kubectl run iperf-server --image=networkstatic/iperf3 -- iperf3 -s kubectl run iperf-client --image=networkstatic/iperf3 -- iperf3 -c IPERF_SERVER_IP Step 2: Check CNI overhead VXLAN: measure encap overhead vs direct routing iptables: check number of rules (iptables-save | wc -l) Step 3: Check conntrack table sysctl net.netfilter.nf_conntrack_count sysctl net.netfilter.nf_conntrack_max # If count approaches max: connection tracking exhaustion
```

CHAPTER 17

Hands-On Exercises

Exercise 6.1 -- Trace End-to-End Packet Flow

Trace every hop of a cross-node request:

```
# Deploy two pods on different nodes: kubectl run pod-a --image=nicolaka/netshoot -- sleep
3600 kubectl run pod-b --image=nginx:alpine # Verify they are on different nodes: kubectl get
pods -o wide # Get Pod-B IP: POD_B_IP=$(kubectl get pod pod-b -o jsonpath=.status.podIP) #
From Pod-A, test connectivity: kubectl exec -it pod-a -- curl -v http://$POD_B_IP # On Pod-A
node: trace routing: ip route get $POD_B_IP # Shows: via flannel.1 or calico interface or
direct # Capture VXLAN traffic (if overlay): tcpdump -i flannel.1 -n host $POD_B_IP -w
/tmp/capture.pcap # Then replay request and inspect encapsulation
```

Exercise 6.2 -- NetworkPolicy Enforcement

Apply default-deny and verify enforcement:

```
# Create isolated namespace: kubectl create namespace netpol-test # Deploy frontend and
backend: kubectl run frontend -n netpol-test --image=nicolaka/netshoot -- sleep 3600 kubectl
run backend -n netpol-test --image=nginx:alpine kubectl expose pod backend -n netpol-test
--port=80 # Verify connectivity (should work before policy): kubectl exec -n netpol-test
frontend -- curl -s http://backend # Apply default-deny: kubectl apply -n netpol-test -f -
<backend: # (apply allow policy; verify connectivity restored)
```

End of Part VI -- Continue to Part VII: Storage

Part VII covers the complete Kubernetes storage stack: CSI architecture and driver development, dynamic provisioning, volume snapshots and cloning, distributed storage systems (Ceph, Longhorn, OpenEBS), cloud storage integrations (EBS, GCS, Azure Disk), backup and disaster recovery strategies, storage performance optimisation, and storage patterns for AI artifacts: model repositories, vector databases, training datasets, and checkpoint management.